

INO Security Audit III

Full re-verification after the latest fix round. Every Critical below was re-read in the code by hand. The short version: the security work was real and five previous Criticals are genuinely closed, but a single commit undid four fixes, and the flagship one still does not execute.

4 Sep 2026 d14447b → fc3aaa5 · 5 commits 494 of 494 tests pass

OVERALL SCORE

5/10

unchanged from 5/10 · was 4 at pass 1

Not ready to submit

Real gains, offset almost exactly by regressions from one commit and three new defects.

Backend & RLS		6 ▲1
Auth & session		6 ▼1
Crypto & local data		5 ▲1
Sharing		5 ▼2
Platform & build		5 ▼2
Privacy & Play		4 ▼1

Start here: one commit undid four fixes

The most recent commit, `fc3aaa5` ("changes"), reverted work from earlier in this same round. It looks like a branch or merge that carried an older version of these files forward. This is the cheapest thing on the list to fix and it removes a Play blocker.

1. **Package ID reverted** to `com.example.inoapp`, which Play rejects outright. iOS is still `com.ino.app`, so the two platforms now disagree.
2. **The debug keystore is committed again.** It had been untracked; this commit re-added it as a binary blob.
3. **The Google terms-consent gate was deleted** from the login screen, orphaning the consent screen entirely.
4. `google-services.json` package name reverted to match.

Verified by checking each of the five commits in turn: the four before it all carried `com.ino.app`. Reverting this one commit's changes to those four files recovers all of it.

What genuinely got fixed

All five Criticals from the last report are properly closed. I re-read each in the code rather than trusting the commit messages.

- **The data-destroying migration is neutered.** The government-ID file is now an explicit no-op with a comment stating that real ID numbers must never be irreversibly overwritten. Safe to run.
- **Offline document encryption is now correct.** Per-user 256-bit key from a secure random source held in Keystore-backed storage, a fresh random nonce for every file, the old hardcoded key demoted to a decrypt-only path with lazy migration, and decryption now throws instead of returning the input.
- **ID numbers are stored in full again, with a proper reveal.** Masking moved from the write path to the render path, the reveal is gated on a forced fresh biometric prompt and auto-hides after 30 seconds, copy yields the real value, and device-PIN fallback means nobody is locked out of their own data. This is the best work in the round.
- **Pass-the-hash is dead at both ends** — bcrypt-only comparison, plus a migration that retires the legacy rows, which was the missing half last time.

- **The rate limiter no longer trusts a client-settable header**, and the password lockout moved into Postgres with a correctly atomic upsert.
- **Deep-link plumbing was fixed carefully on all four fronts** — matching hosts on both platforms, entitlements wired into all three build configurations, and the well-known files present with correct content types.
- **Every test passes** — 494 of 494, up from 461 of 462. The stale assertion was cleaned up too.

Critical

C1 **Account deletion still fails, and now destroys files first** **CRITICAL** **verified**
`supabase/migrations/20260904020000_delete_account_rpc.sql:138,142,146,150,155,171`

The seven wrong object names from last time were fixed. They were replaced by four wrong *column* names. The function queries `document_shares.auth_user_id`, but that table's owner column is `owner_id`. Same mistake on `view_once_shares.vault_documents` uses `shared_by`, not `auth_user_id`. So the function aborts around statement 31 and the whole transaction rolls back — `DELETE FROM auth.users` is never reached and nothing is deleted.

This is now worse than a no-op. The client deletes every one of the user's storage files *before* calling the RPC. So the user taps Delete Account, loses all their uploaded documents, sees an error, and their account and every database row survive. That is destructive partial deletion.

Fix Correct the four column names, then verify by actually deleting a test account and confirming the auth user is gone — not by checking the function exists. Two separate lower-severity problems sit behind it: the storage-object delete compares a uuid to text so it can never match, and it is wrapped in a catch-all that hides the failure.

C2

"Forgot passphrase" wipes the whole vault while reporting zero entries at risk **CRITICAL** **NEW** verified

password_store.dart:47 · local_collection_store.dart:152 · vault_passphrase_sheet.dart:161,171

I traced every link. Tapping "Forgot passphrase" raises a biometric prompt, which drives the app to `inactive`. Two separate lifecycle observers then lock the vault, which clears the in-memory entry list. But that clear does not reset the "already loaded" flag, so the subsequent reload call returns immediately and the list stays empty.

The confirmation dialog then interpolates the entry count and displays **0**, so the user consents believing nothing is at risk. The reset replaces the salt irreversibly, and the resync treats an empty local list as authoritative and deletes every password row on the server. The local copy is still sealed under a key whose salt is now gone. Every password is unrecoverable.

Fix Reset the loaded flag in the memory-clear so the reload actually reloads, and refuse to run the orphan sweep when the local list is empty. Either change alone breaks the chain.

C3

A Keystore hiccup silently destroys the entire offline library **CRITICAL** **NEW** verified

lib/services/offline_document_store.dart:459-481

The new per-user key logic reads before it writes, which is right. But the read is wrapped in a bare `catch (_) {}`, so a *throwing* read is indistinguishable from "no key yet" and execution falls straight through to generate a new key and overwrite the old one. On Android this is not hypothetical: the encrypted store throws after a lock-screen credential change or a backup restore.

One such event replaces the key, and every previously saved offline document fails both the primary decrypt and the legacy fallback. The viewer returns null with no error shown. The same swallow on the write path means a file can be encrypted with a key that was never persisted.

Fix Distinguish "read threw" from "no key present" and never overwrite on the former. Surface the failure rather than silently re-keying.

High

H1

Terms consent is now recorded for nobody HIGH REGRESSION verified

`lib/screens/auth/login_screen.dart` (gate removed in `fc3aaa5`)

The Google consent gate was deleted. The consent screen and the two methods that record acceptance now have zero callers anywhere in the app — I grepped to confirm. The email sign-up path never persisted consent either; it only sets a local boolean. So the consent audit table is empty for every user on every path, and the 18-plus attestation users tick is recorded nowhere.

Fix Restore the gate, and move the check into the shared post-auth routing so both sign-in paths and cold start are covered rather than just the Google button.

H2

Anyone with the public key can reset the share password lockout HIGH

NEW verified

`supabase/migrations/20260904080000_share_security_hardening.sql:129-130`

The new lockout functions are granted to the service role but never revoked from PUBLIC, and Postgres grants execute to PUBLIC by default. So they are callable with the anon key that ships inside the app. An attacker brute-forcing a share calls the record-attempt function with success set to true after each guess, which deletes their counter row, making the five-attempt lockout unlimited.

Fix Add `revoke all ... from public` for both functions. Two other migrations in the same round do this correctly — this one just omits it.

H3

The legacy-hash cleanup turned protected shares into open ones HIGH

NEW verified

`20260904080000_share_security_hardening.sql:20-23` · `functions/share/index.ts:541`

The migration sets `password_hash` to null for every non-bcrypt row. But the edge function treats an empty hash as "no password required" and returns true immediately. So every still-active legacy share that had a password is now viewable by anyone holding the link, for the rest of its lifetime. The replay attack was already closed by rejecting non-bcrypt hashes; this step removed the lock instead of the door.

Fix Revoke or expire those shares instead of nulling the hash.

H4

The two-factor policies are cancelled out by the ones they were meant to replace **HIGH**

supabase/migrations/20260904010000_mfa_rls_hardening.sql:26-45

The helper function was fixed and now uses the correct column. But the new policies were created under different names alongside the original factory-generated ones, which are still present and carry no two-factor condition. Row-level security combines permissive policies with OR, so the effective rule collapses back to plain ownership. A user with an enrolled factor still reads their whole password vault over a single-factor session.

Fix Drop the original policies by their real names, or add the condition to them rather than creating parallel ones.

H5

Deep links still cannot verify, because the package ID no longer matches

HIGH verified

build.gradle.kts:25,45 vs both assetlinks.json

All four deep-link sub-fixes landed correctly. Then the package revert broke the one thing that ties them together: Android verifies a link by matching package name and signing fingerprint, and both fingerprint files declare `com.ino.app` while the app now builds as `com.example.inoapp`. Share links will open the browser, not the app. The iOS half is separately inert because the Apple association file omits the Team ID prefix its format requires.

Fix Comes free with reverting the package change, plus prefixing the Apple entry with your Team ID.

H6

The share web frontend does not compile **HIGH** **NEW** verified

share-frontend/app/s/[token]/DocViewer.tsx:3,152

The file calls `useMemo` but imports only `useEffect`, `useRef` and `useState`. TypeScript strict mode is on and build errors are not suppressed, so every deploy of the share site fails. If a stale build is live, the password-unlock and PDF-view path has never been exercised.

Fix Add `useMemo` to the import. One word.

H7

The live session is still plaintext on disk **HIGH** verified

lib/main.dart:45-49

Unchanged across all three audits. `Supabase.initialize` still has no storage override, so the access and refresh tokens sit as plaintext JSON in shared preferences. The saved-accounts list was moved into Keystore, which makes this more conspicuous rather than less: the secondary copy was protected and the primary one left behind.

H8

A vault admin can still depose the owner HIGH

20260731000000_family_vault_invitations.sql:346-348

Flagged in both prior audits, untouched again. Two rounds have tightened the insert policy; the update policy still lets an admin set the owner's row to a lower role, because its USING clause does not exclude that row. The vault is left ownerless and the guarded promotion RPC cannot undo it.

Also open

AREA	ISSUE	SEVERITY
Backend	The invite function returns a uniform message but still writes the target's name, email and phone into a row the caller reads straight back — a worse leak than the enumeration it replaced.	High
Backend	The storage quota trigger still selects a non-existent <code>size</code> column outside any error handler, which would reject every upload if applied.	High
Backend	Three pairs of migrations share identical timestamps; the CLI keys on that value, so a push would fail on the duplicate.	Medium
Backend	Pre-existing bad vault-document rows are still not cleaned up, so old cross-user file access persists despite the new trigger.	Medium
Backend	Recipient IP addresses are now stored in plaintext with no retention, no purge and no deletion path. The view-once path hashes them; this one does not.	Medium
Auth	The two-factor screen is still dismissable with the Android back button on the account-switch path, and that path still pushes two of them.	Medium
Auth	"Sign out other devices" still only flips a flag the remote device must cooperatively read. No token is revoked.	Medium
Crypto	Writing to the vault while it is locked still stores plaintext, and the sealed-state flag is not modelled, so entries can be corrupted on a cold start.	Medium
Crypto	Decrypted documents are written to the temp root and never deleted — the cleanup routine targets a different directory.	Medium

Sharing	Received documents still persist in the cache after expiry or revocation; the view-once screen cleans up, this one does not.	Medium
Sharing	The pdf.js worker still loads from a third-party CDN, and the site's own policy still allows it alongside unsafe-inline and unsafe-eval.	Medium
Sharing	The web proxies do not forward the visitor's IP, so all web visitors share one lockout bucket — five bad guesses locks out the legitimate recipient.	Medium
Platform	CI still publishes a debug-signed APK as the latest release, through unpinned third-party actions with write access — and the signing key is in the repo again.	Medium
Platform	iOS still compiles out the microphone permission the app requests, so voice navigation cannot work there.	Medium
Privacy	No legal URL is asserted live. The documents cite one host, the app config another, and the README records the first as non-resolving.	Play blocker
Privacy	The in-app policy is unchanged — still six thin sections dated July 2026, contradicting the published markdown, with no link to it.	High
Privacy	The web deletion page still has no form and no API route, while the document describes a verification-code flow that does not exist.	High
Privacy	The data export still returns the user a masked version of their own ID number, and omits most of their data.	Medium
Privacy	Cloud speech is still the default with no caller ever requesting on-device, and voice audio is still absent from the policy.	Medium
Privacy	Creating a reminder unconditionally re-registers push, silently reversing a user who turned notifications off.	Medium

Why the score did not move

Five Criticals closed is real progress, and the identity-reveal work is the best-executed change across all three audits. But three new Criticals appeared, one commit reverted four fixes, and the single most important deliverable — account deletion — failed for the third consecutive attempt, this time with wrong column names instead of wrong table names.

The pattern from the last report has not changed: **fixes are verified by presence rather than by behaviour**. A deletion function that exists but aborts. Two-factor policies that are created but cancelled out. A lockout that is implemented but resettable by anyone. A hash cleanup that runs but unlocks the shares it was protecting. Each would have been caught in minutes by exercising the control once.

The concrete suggestion: for each of the five items below, write down what you would observe if it worked, then check that. Delete a test account and confirm the auth user is gone. Try a sixth password guess after five failures. Open a legacy share that had a password.

What to do next

1 Revert the four file changes in the last commit

Package ID, keystore, consent gate, google-services. Recovers a Play blocker, the deep links and the consent record in one step.

2 Fix the four column names in the deletion function

Then delete a real test account end to end and confirm the auth user is gone.

3 Break the forgot-passphrase chain

Reset the loaded flag on memory-clear, and refuse the orphan sweep on an empty list. Both are one-liners; either one is sufficient.

4 Stop the offline key from being overwritten on a read error

Distinguish a thrown read from an absent key. This is silent total data loss on a common Android event.

5 Revoke the two new lockout functions from PUBLIC, and re-lock the legacy shares

Two SQL lines. Expire them rather than nulling the hash.

6 Drop the original vault policies so the two-factor ones apply

And add `useMemo` to the DocViewer import so the share site builds.

7 Pick one domain, deploy it, and make every document agree

Then rewrite the in-app policy to match the published one and link out to it. This is the remaining Play blocker after the package ID.

Items 1 through 6 are mostly single-line changes and would put this around 7 out of 10. Item 7 is the difference between a submission that is reviewed and one that is rejected before anyone reads the policy.

Method: four parallel re-audits covering backend and row-level security, auth and session, cryptography and local data, sharing and platform, and privacy and Play compliance. Every Critical and High was then independently re-read in the code before inclusion, and several agent claims were checked against the defining schema and rejected or confirmed on that basis. The regressing commit was identified by walking the package identifier back through each of the five commits. The Android build was verified by running the manifest merger, and the full test suite was run. No files were modified during this audit.